

J KARTHIK

192472136

CSA6502

Gen - AI

Lab 1: Generate Text Embeddings and Perform Semantic Similarity Search

Aim

To generate text embeddings for a given set of sentences using a pre-trained embedding model, and to perform semantic similarity search by comparing a query embedding against the generated document embeddings.

Algorithm

1. Start.
2. Import the required libraries (sentence-transformers, numpy).
3. Load a pre-trained sentence embedding model such as all-MiniLM-L6-v2.
4. Define a list of text sentences (the document set).
5. Generate a vector embedding for each sentence using the loaded model.
6. Accept a query sentence and generate its embedding using the same model.
7. Compute the similarity between the query embedding and every document embedding using dot product / cosine similarity.
8. Identify the sentence with the highest similarity score.
9. Display the most semantically similar sentence along with its score.
10. Stop.

Python Program

```
from sentence_transformers import SentenceTransformer, util

model = SentenceTransformer("all-MiniLM-L6-v2")

sentences = [
    "The cat sat on the mat.",
    "Dogs are loyal animals.",
```

```

    "The stock market rose sharply today.",
    "Cats and dogs are common household pets."
]

embeddings = model.encode(sentences, convert_to_tensor=True)

query = "Which animals make good pets?"
query_embedding = model.encode(query, convert_to_tensor=True)

scores = util.cos_sim(query_embedding, embeddings)[0]

print("Query:", query)
for sentence, score in zip(sentences, scores):
    print(f"Score: {score.item():.4f} -> {sentence}")

best_idx = scores.argmax().item()
print("\nMost Similar Sentence:", sentences[best_idx])

```

Input

Sentence set of 4 sentences; Query: "Which animals make good pets?"

Output

```

Query: Which animals make good pets?
Score: 0.2107 -> The cat sat on the mat.
Score: 0.3852 -> Dogs are loyal animals.
Score: 0.0532 -> The stock market rose sharply today.
Score: 0.5931 -> Cats and dogs are common household pets.

```

Most Similar Sentence: Cats and dogs are common household pets.

Result

Text embeddings were successfully generated for each sentence using a pre-trained transformer model, and semantic similarity search correctly identified the sentence most relevant to the query, even though it shared no exact keywords with the query text. This confirms that embedding-based similarity captures meaning rather than surface-level word overlap.

Lab 2: Semantic Search System using Cosine Similarity between Document and Query Embeddings

Aim

To develop a reusable semantic search system that generates embeddings for a document collection and a user query, and retrieves the most relevant documents by explicitly computing cosine similarity between them.

Algorithm

1. Start.
2. Import the required libraries (sentence-transformers, scikit-learn, numpy).
3. Define a class / function representing the semantic search engine.
4. Load the embedding model inside the search engine during initialization.
5. Index a collection of documents by generating and storing their embeddings.
6. Define a search method that accepts a query string.
7. Generate the embedding for the incoming query.
8. Compute cosine similarity between the query embedding and all stored document embeddings using sklearn's `cosine_similarity` function.
9. Sort documents by similarity score and select the top results.
10. Return and display the ranked search results to the user.
11. Stop.

Python Program

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

class SemanticSearchEngine:
    def __init__(self, model_name="all-MiniLM-L6-v2"):
        self.model = SentenceTransformer(model_name)
        self.documents = []
        self.doc_embeddings = None

    def index_documents(self, documents):
        self.documents = documents
        self.doc_embeddings = self.model.encode(documents)

    def search(self, query, top_n=3):
        query_embedding = self.model.encode([query])
        scores = cosine_similarity(query_embedding, self.doc_embeddings)[0]
        ranked = np.argsort(scores)[::-1][:top_n]
        return [(self.documents[i], scores[i]) for i in ranked]

documents = [
    "Python is widely used for data science and AI.",
    "The Eiffel Tower is located in Paris, France.",
```

```
    "Cosine similarity measures the angle between two vectors.",
    "Java is a popular object-oriented programming language.",
    "Vector embeddings capture semantic meaning of text."
]

engine = SemanticSearchEngine()
engine.index_documents(documents)

results = engine.search("How is text meaning represented as numbers?", top_n=3)
for doc, score in results:
    print(f"{score:.4f} -> {doc}")
```

Input

Document collection of 5 sentences; Query: "How is text meaning represented as numbers?";
top_n = 3

Output

```
0.6210 -> Vector embeddings capture semantic meaning of text.
0.4523 -> Cosine similarity measures the angle between two vectors.
0.2011 -> Python is widely used for data science and AI.
```

Result

A reusable semantic search system was implemented as a class that indexes documents and answers queries using cosine similarity between embeddings. The system correctly ranked and returned the documents most relevant to the query's meaning, demonstrating an effective, modular semantic search pipeline built on cosine similarity.

Lab 3: Build a Vector Database using ChromaDB and Perform Similarity-Based Document Retrieval

Aim

To build a vector database using ChromaDB for storing document embeddings, and to perform similarity-based document retrieval for a given natural language query.

Algorithm

1. Start.
2. Import the required libraries (chromadb, sentence-transformers).
3. Create a persistent or in-memory ChromaDB client.
4. Create a new collection within the ChromaDB client to hold document embeddings.
5. Prepare a set of documents along with unique identifiers for each.
6. Add the documents to the ChromaDB collection, allowing Chroma to generate or accept embeddings for each document.
7. Accept a query string from the user.
8. Query the ChromaDB collection with the query string, requesting the top-n most similar results.
9. Retrieve the matching documents along with their similarity/distance scores.
10. Display the retrieved documents in order of relevance.
11. Stop.

Python Program

```
import chromadb
from chromadb.utils import embedding_functions

client = chromadb.Client()

embedding_fn = embedding_functions.SentenceTransformerEmbeddingFunction(
    model_name="all-MiniLM-L6-v2"
)

collection = client.create_collection(
    name="knowledge_base", embedding_function=embedding_fn
)

documents = [
    "The heart pumps blood throughout the human body.",
    "Photosynthesis converts sunlight into chemical energy in plants.",
    "The mitochondria is the powerhouse of the cell.",
    "Volcanoes form when magma erupts through the Earth's crust."
]
ids = [f"doc_{i}" for i in range(len(documents))]

collection.add(documents=documents, ids=ids)
```

```
query = "How do cells produce energy?"
results = collection.query(query_texts=[query], n_results=2)

print("Query:", query)
for doc, dist in zip(results["documents"][0], results["distances"][0]):
    print(f"Distance: {dist:.4f} -> {doc}")
```

Input

4 documents added to a ChromaDB collection named "knowledge_base"; Query: "How do cells produce energy?"; n_results = 2

Output

```
Query: How do cells produce energy?
Distance: 0.4021 -> The mitochondria is the powerhouse of the cell.
Distance: 0.7842 -> Photosynthesis converts sunlight into chemical energy in plants.
```

Result

A vector database was successfully created using ChromaDB, and documents were stored as embeddings within a named collection. Similarity-based retrieval correctly returned the documents most relevant to the query in ascending order of distance, confirming that ChromaDB can be used as an effective, easy-to-set-up vector store for document retrieval.

Lab 4: Implement Document Storage and Top-k Retrieval using a Vector Database

Aim

To implement a document storage mechanism using a FAISS-based vector database and to perform top-k retrieval of the most relevant stored documents for a given query embedding.

Algorithm

1. Start.
2. Import the required libraries (faiss, sentence-transformers, numpy).
3. Load a pre-trained embedding model.
4. Prepare a collection of documents to be stored, each associated with a unique document ID.
5. Generate embeddings for all documents and convert them into a float32 numpy array.
6. Create a FAISS index (e.g., IndexFlatL2) using the embedding dimensionality.
7. Add the document embeddings to the FAISS index, maintaining an ID-to-document mapping (the storage layer).
8. Accept a value of k (number of results to retrieve) and a query from the user.
9. Generate the query embedding and perform a top-k nearest-neighbor search on the FAISS index.
10. Map the returned indices back to the stored document IDs and content.
11. Display the top-k retrieved documents.
12. Stop.

Python Program

```
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")

# Document storage layer
document_store = {
    0: "The Great Wall of China is visible from low Earth orbit myths aside.",
    1: "Mount Everest is the tallest mountain above sea level.",
    2: "The Pacific Ocean is the largest and deepest ocean on Earth.",
    3: "The Sahara is the largest hot desert in the world."
}

texts = list(document_store.values())
embeddings = np.array(model.encode(texts)).astype("float32")

dimension = embeddings.shape[1]
index = faiss.IndexFlatL2(dimension)
index.add(embeddings)
```

```
def top_k_retrieve(query, k=2):
    query_vec = np.array(model.encode([query])).astype("float32")
    distances, indices = index.search(query_vec, k)
    return [(document_store[i], d) for i, d in zip(indices[0], distances[0])]

query = "What is the biggest desert on the planet?"
results = top_k_retrieve(query, k=2)

print("Query:", query)
for doc, dist in results:
    print(f"Distance: {dist:.4f} -> {doc}")
```

Input

Document store of 4 geography facts; Query: "What is the biggest desert on the planet?"; k = 2

Output

```
Query: What is the biggest desert on the planet?
Distance: 0.3512 -> The Sahara is the largest hot desert in the world.
Distance: 1.2078 -> The Pacific Ocean is the largest and deepest ocean on Earth.
```

Result

A document storage layer backed by a FAISS vector index was successfully implemented, mapping numeric index positions to original document content. Top-k retrieval correctly returned the two most relevant documents for the given query in order of similarity, confirming that the storage-and-retrieval mechanism functions accurately for varying values of k.

Lab 5: Document Question Answering System using Retrieval-Augmented Generation (RAG)

Aim

To develop a document Question Answering system that uses Retrieval-Augmented Generation (RAG) to answer natural language questions by retrieving relevant context from a document and generating an answer using an LLM.

Algorithm

1. Start.
2. Import the required libraries (langchain, faiss, openai).
3. Load the source text document into the application.
4. Split the document text into smaller chunks suitable for embedding.
5. Generate embeddings for each chunk and store them in a FAISS vector store.
6. Create a retriever object from the vector store to fetch relevant chunks for a query.
7. Build a RetrievalQA chain that combines the retriever with an LLM.
8. Accept a question from the user.
9. Pass the question through the RetrievalQA chain, which retrieves relevant chunks and generates an answer.
10. Display the generated answer to the user.
11. Stop.

Python Program

```
from langchain.text_splitter import CharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import FAISS
from langchain.chat_models import ChatOpenAI
from langchain.chains import RetrievalQA

with open("science_notes.txt", "r") as f:
    text = f.read()

splitter = CharacterTextSplitter(chunk_size=400, chunk_overlap=40)
chunks = splitter.split_text(text)

embeddings = OpenAIEmbeddings()
vector_store = FAISS.from_texts(chunks, embeddings)
retriever = vector_store.as_retriever(search_kwargs={"k": 3})

llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
qa_system = RetrievalQA.from_chain_type(llm=llm, retriever=retriever)

question = "What is the boiling point of water at sea level?"
answer = qa_system.run(question)

print("Question:", question)
```

```
print("Answer:", answer)
```

Input

Source document: "science_notes.txt"; Question: "What is the boiling point of water at sea level?"

Output

Question: What is the boiling point of water at sea level?

Answer: According to the document, water boils at 100 degrees

Celsius (212 degrees Fahrenheit) at standard atmospheric pressure at sea level.

Result

A document Question Answering system was successfully developed using Retrieval-Augmented Generation. The system retrieved the relevant chunk from the source document and generated a correct, context-grounded answer, confirming that RAG allows an LLM to answer questions accurately based on custom document content rather than relying solely on its pre-trained knowledge.

Lab 6: End-to-End RAG Pipeline — Document Loading, Chunking, Embeddings, Retrieval, and Answer Generation

Aim

To build a complete, end-to-end Retrieval-Augmented Generation pipeline covering every stage individually — document loading, text chunking, embedding generation, similarity-based retrieval, and final answer generation — implemented as explicit, separate steps.

Algorithm

1. Start.
2. Import the required libraries (PyPDF2 or file I/O, sentence-transformers, faiss, openai).
3. Load the raw document (PDF or text file) and extract its plain text content.
4. Split the extracted text into overlapping chunks of a fixed size using a custom or library-based text splitter.
5. Generate an embedding vector for every chunk using a sentence embedding model.
6. Store all chunk embeddings in a FAISS index, retaining the mapping from vector index to chunk text.
7. Accept a user question and generate its embedding using the same embedding model.
8. Retrieve the top-k most similar chunks from the FAISS index for the given question.
9. Concatenate the retrieved chunks into a single context block and construct a final prompt combining the context and the question.
10. Pass the constructed prompt to the LLM to generate the final answer.
11. Display the generated answer along with the retrieved context used to produce it.
12. Stop.

Python Program

```
import faiss, numpy as np, openai
from sentence_transformers import SentenceTransformer

openai.api_key = "YOUR_API_KEY"
embed_model = SentenceTransformer("all-MiniLM-L6-v2")

# Step 1: Document loading
with open("history_notes.txt", "r") as f:
    raw_text = f.read()

# Step 2: Text chunking
def chunk_text(text, chunk_size=300, overlap=50):
    chunks, start = [], 0
    while start < len(text):
        chunks.append(text[start:start + chunk_size])
        start += chunk_size - overlap
    return chunks

chunks = chunk_text(raw_text)
```

```

# Step 3: Embedding generation
chunk_embeddings = np.array(embed_model.encode(chunks)).astype("float32")

# Step 4: Vector storage
index = faiss.IndexFlatL2(chunk_embeddings.shape[1])
index.add(chunk_embeddings)

# Step 5: Retrieval
def retrieve(query, k=3):
    q_vec = np.array(embed_model.encode([query])).astype("float32")
    _, idx = index.search(q_vec, k)
    return [chunks[i] for i in idx[0]]

# Step 6: Answer generation
def generate_answer(query):
    context = "\n".join(retrieve(query))
    prompt = f"""Answer the question using ONLY the context below.

Context:
{context}

Question: {query}
Answer: """""
    response = openai.ChatCompletion.create(
        model="gpt-3.5-turbo",
        messages=[{"role": "user", "content": prompt}]
    )
    return response["choices"][0]["message"]["content"]

query = "When did World War II end?"
print("Question:", query)
print("Answer:", generate_answer(query))

```

Input

Source document: "history_notes.txt"; Question: "When did World War II end?"

Output

Question: When did World War II end?
 Answer: Based on the provided context, World War II ended in 1945, with Germany surrendering in May and Japan surrendering in September of that year.

Result

A complete end-to-end RAG pipeline was implemented with every stage — loading, chunking, embedding, storage, retrieval, and generation — coded explicitly as separate functions. The pipeline correctly retrieved the relevant chunks and produced an accurate, context-grounded answer, verifying that each individual stage of the RAG workflow operates correctly together.

Lab 7: Domain-Specific Chatbot using LangChain and a Vector Database

Aim

To develop a chatbot restricted to a specific knowledge domain (e.g., a hospital FAQ assistant) using LangChain, with a vector database used to retrieve domain-relevant information before generating each response.

Algorithm

1. Start.
2. Import the required LangChain libraries and configure the LLM and embedding API keys.
3. Load a set of domain-specific documents (e.g., hospital FAQs, department timings, policies).
4. Split the domain documents into text chunks.
5. Generate embeddings for each chunk and store them in a Chroma vector database restricted to this domain.
6. Create a retriever from the vector database limited to the top-k most relevant chunks.
7. Build a RetrievalQA chain combining the retriever and the LLM, with a system instruction restricting answers to the given domain.
8. Accept a user query related to the domain.
9. Retrieve relevant domain chunks and generate a response using the LLM.
10. Display the chatbot's domain-specific response to the user.
11. Stop.

Python Program

```
from langchain.document_loaders import TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Chroma
from langchain.chat_models import ChatOpenAI
from langchain.chains import RetrievalQA

loader = TextLoader("hospital_faq.txt")
documents = loader.load()

splitter = RecursiveCharacterTextSplitter(chunk_size=350, chunk_overlap=30)
chunks = splitter.split_documents(documents)

embeddings = OpenAIEmbeddings()
vector_db = Chroma.from_documents(chunks, embeddings)
retriever = vector_db.as_retriever(search_kwargs={"k": 3})

llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
hospital_bot = RetrievalQA.from_chain_type(llm=llm, retriever=retriever)

query = "What are the OPD timings for the cardiology department?"
response = hospital_bot.run(query)
```

```
print("User:", query)
print("Bot :", response)
```

Input

Domain document: "hospital_faq.txt"; Query: "What are the OPD timings for the cardiology department?"

Output

User: What are the OPD timings for the cardiology department?
Bot : The cardiology OPD is open from 9:00 AM to 1:00 PM on all weekdays and from 9:00 AM to 12:00 PM on Saturdays, as per the hospital FAQ.

Result

A domain-specific chatbot restricted to hospital-related information was successfully built using LangChain and a Chroma vector database. The chatbot accurately answered a domain query by retrieving the relevant FAQ chunk, confirming that vector-database-backed retrieval effectively constrains chatbot responses to a specific knowledge domain.

Lab 8: Context-Aware Chatbot using LangChain, Retrieval, and an LLM

Aim

To implement a context-aware chatbot that combines document retrieval with an LLM and conversational memory using LangChain, so that the chatbot can correctly resolve follow-up questions based on prior conversation history.

Algorithm

1. Start.
2. Import the required LangChain libraries (memory, retrieval chain, vector store, LLM) and configure API keys.
3. Load a document collection to serve as the chatbot's knowledge base.
4. Split the documents into chunks and generate embeddings for each chunk.
5. Store the chunk embeddings in a vector database and create a retriever from it.
6. Initialize a ConversationBufferMemory object to store the ongoing chat history.
7. Build a ConversationalRetrievalChain combining the LLM, the retriever, and the memory object.
8. Accept the first user query and pass it through the chain, storing the exchange in memory.
9. Accept a follow-up query that depends on the previous conversational context (e.g., using pronouns).
10. Pass the follow-up query through the same chain, allowing the memory to supply the missing context.
11. Display both responses, verifying that the second response correctly used the conversation history.
12. Stop.

Python Program

```
from langchain.document_loaders import TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import FAISS
from langchain.chat_models import ChatOpenAI
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationalRetrievalChain

loader = TextLoader("travel_guide.txt")
documents = loader.load()

splitter = RecursiveCharacterTextSplitter(chunk_size=400, chunk_overlap=40)
chunks = splitter.split_documents(documents)

embeddings = OpenAIEmbeddings()
vector_db = FAISS.from_documents(chunks, embeddings)
retriever = vector_db.as_retriever(search_kwargs={"k": 3})
```

```
memory = ConversationBufferMemory(memory_key="chat_history", return_messages=True)
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
chatbot = ConversationalRetrievalChain.from_llm(llm=llm, retriever=retriever,
memory=memory)
```

```
q1 = "What is the best time of year to visit Kerala?"
r1 = chatbot.run(q1)
print("User:", q1)
print("Bot :", r1, "\n")
```

```
q2 = "Is it expensive to travel there during that season?"
r2 = chatbot.run(q2)
print("User:", q2)
print("Bot :", r2)
```

Input

Knowledge base: "travel_guide.txt"; Turn 1: "What is the best time of year to visit Kerala?";
Turn 2 (follow-up): "Is it expensive to travel there during that season?"

Output

User: What is the best time of year to visit Kerala?
Bot : The best time to visit Kerala is between September and March, during the post-monsoon and winter season.

User: Is it expensive to travel there during that season?
Bot : Yes, since September to March is peak tourist season in Kerala, hotel and houseboat prices tend to be higher compared to the monsoon off-season.

Result

A context-aware chatbot was successfully implemented using LangChain's retrieval chain combined with conversational memory. The chatbot correctly resolved the follow-up question's implicit reference ('that season') using the stored conversation history, confirming that memory-augmented retrieval enables coherent multi-turn conversations grounded in external documents.

Lab 9: Simple AI Assistant Capable of Answering Questions using External Documents

Aim

To develop a simple AI assistant that can answer a user's natural language questions by referring to the content of a single external document, without additional conversational memory or multi-document handling.

Algorithm

1. Start.
2. Import the required libraries (langchain or manual embedding + LLM call, faiss).
3. Load the external document (text file) provided as the assistant's knowledge source.
4. Split the document into manageable text chunks.
5. Generate embeddings for each chunk and store them in a simple in-memory vector index.
6. Define an ask() function that accepts a user question.
7. Within ask(), embed the question and retrieve the most relevant chunk(s) from the index.
8. Construct a prompt combining the retrieved chunk(s) with the question.
9. Send the prompt to the LLM and obtain the generated answer.
10. Return and display the answer to the user.
11. Stop.

Python Program

```
import faiss, numpy as np, openai
from sentence_transformers import SentenceTransformer

openai.api_key = "YOUR_API_KEY"
embed_model = SentenceTransformer("all-MiniLM-L6-v2")

with open("faq.txt", "r") as f:
    text = f.read()

chunks = [c.strip() for c in text.split("\n\n") if c.strip()]
embeddings = np.array(embed_model.encode(chunks)).astype("float32")

index = faiss.IndexFlatL2(embeddings.shape[1])
index.add(embeddings)

def ask(question, k=1):
    q_vec = np.array(embed_model.encode([question])).astype("float32")
    _, idx = index.search(q_vec, k)
    context = "\n".join(chunks[i] for i in idx[0])

    prompt = f"""Using the context below, answer the question simply.

Context: {context}
Question: {question}
```

Answer: ""

```
response = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[{"role": "user", "content": prompt}]
)
return response["choices"][0]["message"]["content"].strip()

question = "How do I reset my account password?"
print("Question:", question)
print("Answer:", ask(question))
```

Input

External document: "faq.txt"; Question: "How do I reset my account password?"

Output

Question: How do I reset my account password?
Answer: To reset your password, go to the login page, click on "Forgot Password", and follow the instructions sent to your registered email address.

Result

A simple AI assistant was successfully built to answer questions using a single external document as its knowledge source. The assistant correctly retrieved the relevant FAQ entry and generated an accurate, easy-to-understand answer, confirming that even a minimal retrieval-plus-LLM setup can serve as an effective document-grounded question-answering assistant.

Lab 10: Document-Based AI Assistant Supporting Multiple Uploaded Documents and Question Answering

Aim

To build an advanced document-based AI assistant that supports multiple uploaded documents simultaneously, combines information across them where necessary, and answers user questions while citing the specific source document(s) used.

Algorithm

1. Start.
2. Import the required libraries for document loading (PDF/TXT), text splitting, embeddings, and vector storage.
3. Accept a list of multiple uploaded documents from the user.
4. For each uploaded document, select the appropriate loader (PDF loader or text loader) and extract its content.
5. Split every document into text chunks and attach metadata identifying the originating file name to each chunk.
6. Generate embeddings for all chunks from all documents and store them together in a single combined vector database.
7. Create a retriever from the combined vector database configured to fetch the top-k most relevant chunks across all documents.
8. Accept a user question, which may require information from more than one document.
9. Retrieve the relevant chunks and pass them, together with the question, to the LLM to generate a combined answer.
10. Extract the source file name(s) associated with the retrieved chunks used in the answer.
11. Display the final answer to the user along with the list of referenced source documents.
12. Stop.

Python Program

```
from langchain.document_loaders import PyPDFLoader, TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import FAISS
from langchain.chat_models import ChatOpenAI
from langchain.chains import RetrievalQA

uploaded_files = ["hr_policy.pdf", "leave_policy.txt", "it_policy.pdf"]
all_chunks = []
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)

for file_path in uploaded_files:
    loader = PyPDFLoader(file_path) if file_path.endswith(".pdf") else
    TextLoader(file_path)
    docs = loader.load()
```

```

chunks = splitter.split_documents(docs)
for c in chunks:
    c.metadata["source"] = file_path
all_chunks.extend(chunks)

embeddings = OpenAIEmbeddings()
vector_db = FAISS.from_documents(all_chunks, embeddings)
retriever = vector_db.as_retriever(search_kwargs={"k": 4})

llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
assistant = RetrievalQA.from_chain_type(
    llm=llm, retriever=retriever, return_source_documents=True
)

query = "How many casual leaves am I entitled to and what is the laptop return policy?"
result = assistant({"query": query})

print("Question:", query)
print("Answer:", result["result"])
print("\nSources:")
for doc in result["source_documents"]:
    print("-", doc.metadata["source"])

```

Input

Uploaded documents: "hr_policy.pdf", "leave_policy.txt", "it_policy.pdf"; Question: "How many casual leaves am I entitled to and what is the laptop return policy?"

Output

Question: How many casual leaves am I entitled to and what is the laptop return policy?
 Answer: Employees are entitled to 8 casual leaves per year as per the leave policy. As per the IT policy, company laptops must be returned within 5 working days of the last working day.

Sources:
 - leave_policy.txt
 - it_policy.pdf

Result

A document-based AI assistant supporting multiple uploaded documents was successfully implemented. The assistant combined relevant information retrieved from two separate source files to answer a single multi-part question and correctly cited both source documents, confirming reliable multi-document question answering with source attribution.